K.RAMAKRISHNAN COLLEGE OF TECHNOLOGY

(Autonomous)

SAMAYAPURAM, TRICHY-621 112

Incubating Minds,
Catalyzing Careers

DEPARTMENT OF INFORMATION TECHNOLOGY

Practical Record Note

Name : KAVIYA V

Register No 2503811720522022

Course code : CGI1242

Course name : Data Science and Statistical Analysis laboratory

Branch : B.Tech – Information Technology

K.RAMAKRISHNAN COLLEGE OF TECHNOLOGY

(Autonomous)

SAMAYAPURAM, TRICHY-621 112

Incubating Minds,
Catalyzing Careers

CERTIFICATE

Certified that this is a bonafide record of work done by KAVIYA V of 3rd Semester in CGI1242 Data Science and Statistical Analysis laboratory during the academic year 2026-2027

her University Register Number is 2503811720522022

Staff In-charge: **Head of the Department**

Date: Date:

Submitted for the practical exam held on:

Internal Examiner External Examiner

Date: Date:

VISION OF DEPARTMENT:

To be a leading force in leveraging cutting-edge technology to drive students with innovation, efficiency, and strategic growth.

MISSION OF DEPARTMENT:

M1: Academic Excellence: To provide high-quality education that equips students with the theoretical knowledge and practical skills necessary to excel in their careers and adapt to the evolving technological landscape.

M2: Innovative Research: To foster a research-oriented environment that encourages innovation, interdisciplinary collaboration, and the pursuit of cutting-edge solutions to complex problems in technology and related fields.

M3: Ethical and Inclusive Education: To promote ethical practices, diversity, and inclusion within the IT field, preparing students to be responsible and culturally aware professionals.

PROGRAM EDUCATIONAL OBJECTIVES:

PEO1: Domain Knowledge: To produce graduates who have strong foundation of knowledge and skills in the field of Information Technology.

PEO2: Employability Skills and Research: To produce graduates who are employable in industries/ public sector/ research organizations or work as an entrepreneur.

PEO3: Ethics and Values: To develop leadership skills and ethically collaborate with society to tackle real-world challenges.

PROGRAM OUTCOMES:

Engineering Graduates will be able to:

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the

public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.(WK8)

PROGRAMME SPECIFIC OUTCOME

PSO 1: System Design and Development: Demonstrate the ability to apply essential IT and computing principles to develop robust and efficient software solutions.

PSO 2: Data Communication and Networking: Analyze and evaluate system resources to ensure scalability, and optimal performance in distributed and networked environments.

PSO 3: Computational and Data Intelligence: Design and deployment of intelligent systems to uncover hidden patterns in complex datasets that make data-driven decisions

COURSE OUTCOME

CO1	Demonstrate their ability to implement fundamental cryptographic algorithms.
CO2	Implement symmetric and asymmetric encryption algorithms and apply them to secure messages in a network environment.
CO3	Implement and analyze message authentication codes and digital signatures to ensure the authenticity and integrity of messages.
CO4	Set up and configure honey pots using KF Sensor, deploying the mina network environment to attract and analyze traffic
CO5	Configure and use Net Stumbler to discover and analyze available wireless networks, including those using WEP and WPA encryption.

MAPPING OF COURSE OUTCOMES WITH PROGRAMME OUTCOMES

CO	Programme Outcomes (PO) & Programme Specific Outcomes (PSO)													
	PO1	**PO2**	**PO3**	**PO4**	**PO5**	**PO6**	**PO7**	**PO8**	**PO9**	**PO10**	**PO11**	**PSO1**	**PSO2**	**PSO3**
CO1	3	3	1	2	3			1			2	3	2	1
CO2	3	2	1	1	3			1			1	3	2	1
CO3	3	2	3	2	3			1		1	2	3	2	1
CO4	3	2	3	2	3	1		1		1	2	3	2	1
CO5	3	2	3	2	3	1		1		1	2	3	2	1

Incubating Minds,
Catalyzing Careers

K.Ramakrishnan
College of Technology
Autonomous
Affiliated to Anna University Chennai, Approved by AICTE New Delhi,
Accredited by NBA and with 'A+' grade by NAAC
Samayapuram, Tiruchirappalli - 621 112, Tamilnadu, India.

Lab Assessment Rubrics

Criteria	Excellent	Good	Satisfactory	Unsatisfactory
Record (15 marks)				
Program Execution (5)	Program runs correctly with expected output and clear logic.. (5)	Program runs with minor errors or partial output; logic mostly correct. (3-4)	Major logic/syntax errors; program does not compile or run meaningfully. (1-2)	No attempt made or program is irrelevant/incomplete. (0)
Screenshots and Output Evidence (5)	Clear, complete screenshots for all steps, properly labeled (5)	Few steps missed or minor labeling issues (3-4)	Incomplete or unclear screenshots (1-2)	Missing or irrelevant screenshots (0)
Presentation & Timely Submission (5)	Well-formatted, neat, easy to read, and submitted on or before the deadline. (5)	Mostly neat with minor formatting issues, submitted late by 1 day. (3-4)	Average formatting with some clutter or inconsistencies, submitted late by 2 days. (1-2)	Poorly presented and disorganized, and not submitted. (0)
Viva (10 marks)				
Knowledge & Understanding (6)	Comprehensive and accurate understanding demonstrated during viva. (5-6)	Good understanding, with some minor gaps or inaccuracies. (3-4)	Adequate understanding, with noticeable gaps or inaccuracies. (1-2)	Poor understanding, with critical gaps or inaccuracies. (0)
Communication Skills (4)	Clear, concise, and effective communication during viva. (4)	Mostly clear communication with minor issues. (3)	Communication is understandable but may lack clarity at times. (1-2)	Communication is unclear, disorganized, or incoherent. (0)

EXP.NO	DATE	NAME OF THE PROGRAM	PAGE NO.
1(A)		Create a Database in excel and convert it into CSV files	
1(B)		Upload the CSV file and Analyse datasets using data mining tool.	
1(C)		Explore the data types and variables and analyze the features	
1(D)		Normalize the datasets using preprocessing techniques using WEKA tool	
2		Install and explore the features of NumPy, SciPy, Jupyter, Statsmodels and Pandas packages.	
3		Working with Numpy Arrays	
4		Working with Pandas data frames	
5		Descriptive Analysis: Reading Data from different files and web sources using Python	
6		Working with Univariate and Bivariate Analysis	

Ex.No. 1a	Create a Database in excel and convert it into CSV files
DATE:10.07.26	

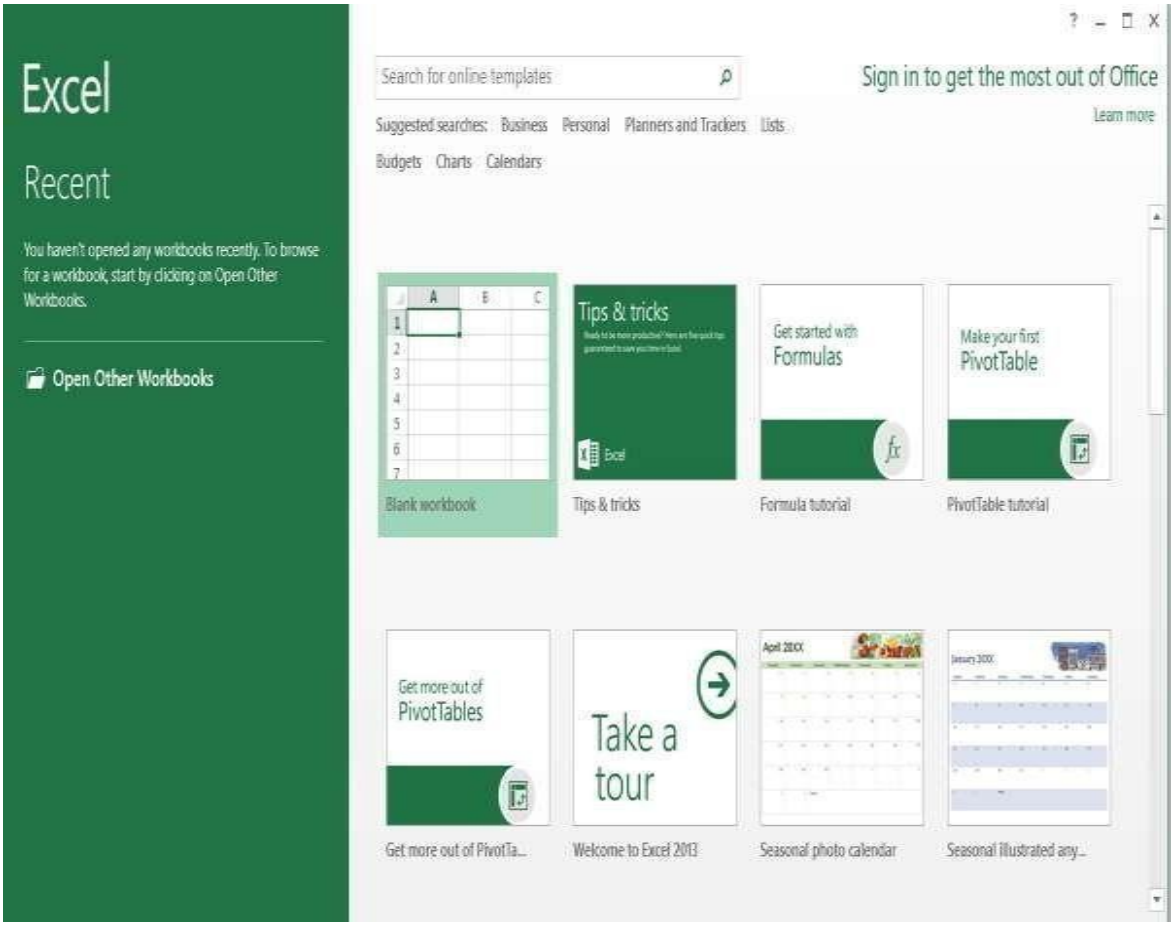
Objective:

To create a database in Microsoft Excel and convert it into a CSV file for further analysis.

Procedure:

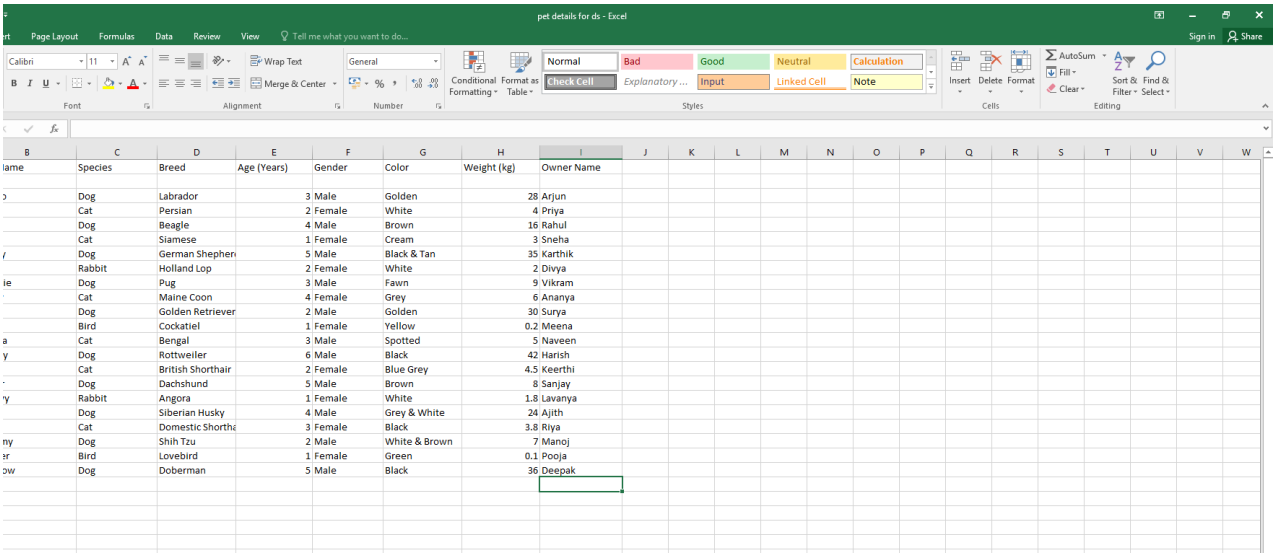
1. Open a Microsoft excel.

2. Create a New workbook. Click on FILE NEW BLANK WORKBOOK



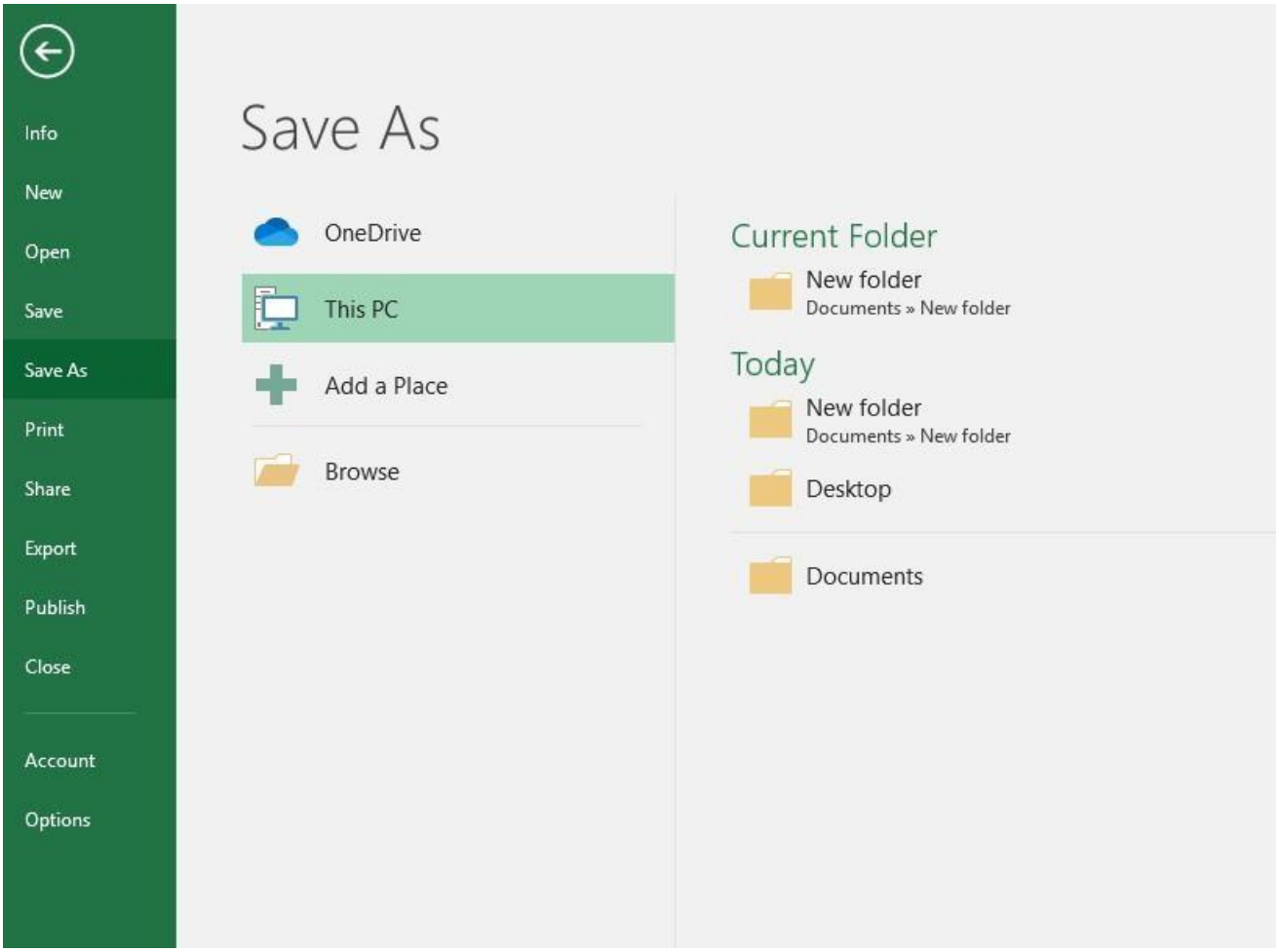
1. Enter the data in the worksheet.

A	B	C	D	E	F	G	H	I
Pet ID	Pet Name	Species	Breed	Age (Years)	Gender	Color	Weight (kg)	Owner Name
P001	Bruno	Dog	Labrador	3	Male	Golden	28	Arjun
P002	Bella	Cat	Persian	2	Female	White	4	Priya
P003	Max	Dog	Beagle	4	Male	Brown	16	Rahul
P004	Luna	Cat	Siamese	1	Female	Cream	3	Sneha
P005	Rocky	Dog	German Shepher	5	Male	Black & Tan	35	Karthik
P006	Coco	Rabbit	Holland Lop	2	Female	White	2	Divya
P007	Charlie	Dog	Pug	3	Male	Fawn	9	Vikram
P008	Daisy	Cat	Maine Coon	4	Female	Grey	6	Ananya
P009	Milo	Dog	Golden Retriever	2	Male	Golden	30	Surya
P010	Lucy	Bird	Cockatiel	1	Female	Yellow	0.2	Meena
P011	Simba	Cat	Bengal	3	Male	Spotted	5	Naveen
P012	Buddy	Dog	Rottweiler	6	Male	Black	42	Harish
P013	Nala	Cat	British Shorthair	2	Female	Blue Grey	4.5	Keerthi
P014	Oscar	Dog	Dachshund	5	Male	Brown	8	Sanjay
P015	Snowy	Rabbit	Angora	1	Female	White	1.8	Lavanya
P016	Leo	Dog	Siberian Husky	4	Male	Grey & White	24	Ajith
P017	Kitty	Cat	Domestic Shortha	3	Female	Black	3.8	Riya
P018	Tommy	Dog	Shih Tzu	2	Male	White & Brown	7	Manoj
P019	Ginger	Bird	Lovebird	1	Female	Green	0.1	Pooja
P020	Shadow	Dog	Doberman	5	Male	Black	36	Deepak



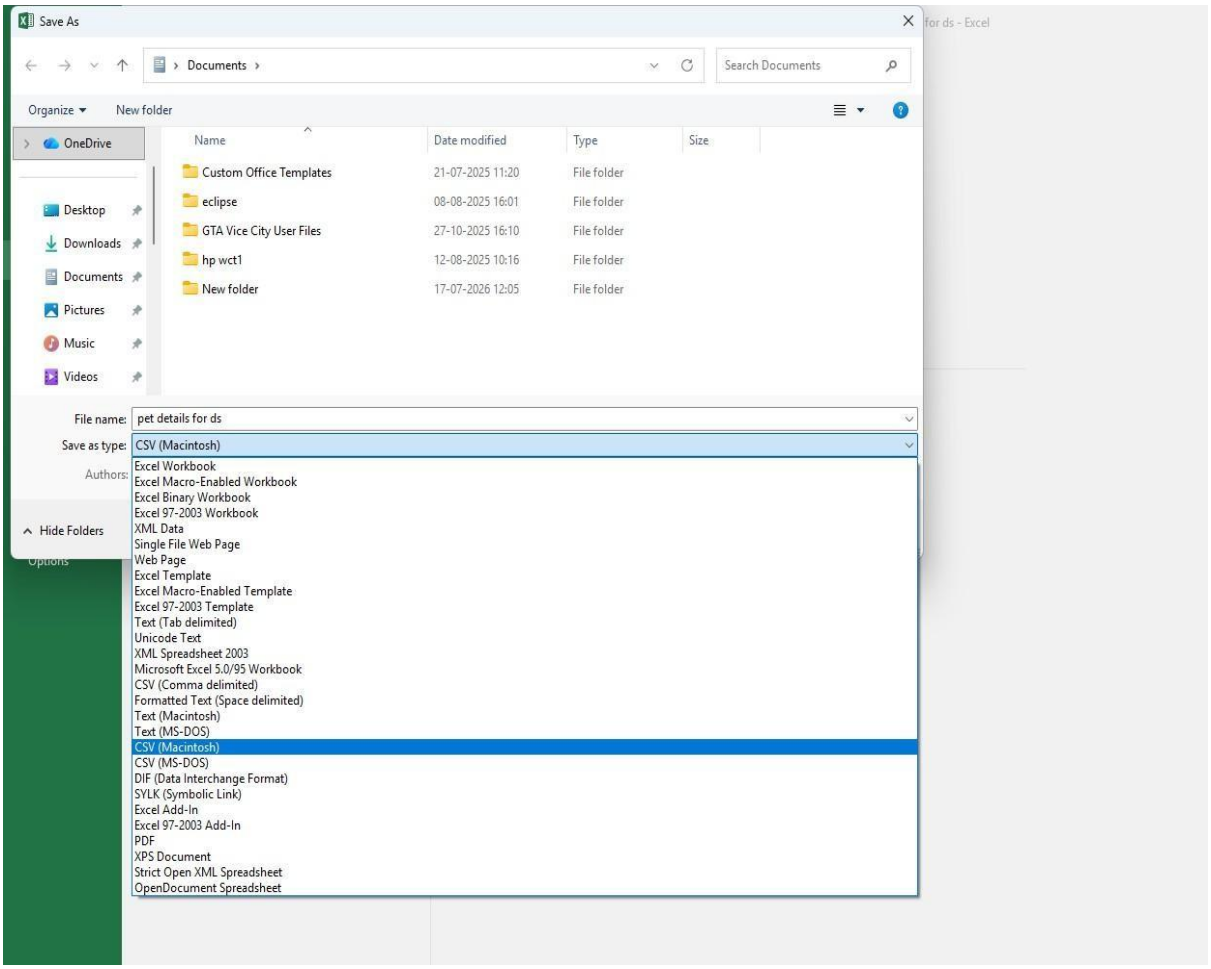
1. Save the excel file

- Click on "File" > "Save As".
- Choose a location to save the file.



2. Enter a name for your file and click "Save".5. Convert the Excel File to CSV

- Save as CSV
- Click on "File" > "Save As".
- Choose the location where you want to save the CSV file.
- In the "Save as type" dropdown, select "CSV (Comma delimited) (*.csv)".
- Enter a name for your CSV file and click "Save".



1. Confirm the CSV file creation

- Confirm the CSV Format
- A warning might pop up stating that some features in your workbook might be lost if you save it as CSV. Click "Yes" to continue.

Conclusion:

The process of creating a database in Excel and converting it into a CSV file was completed successfully.

Ex.No. 1b	The visual graph representation of data
DATE:10.07.26	

Objective:

The process is by using the weka tool how we get the out for the csv file and get the visual graph representation was completed successfully.

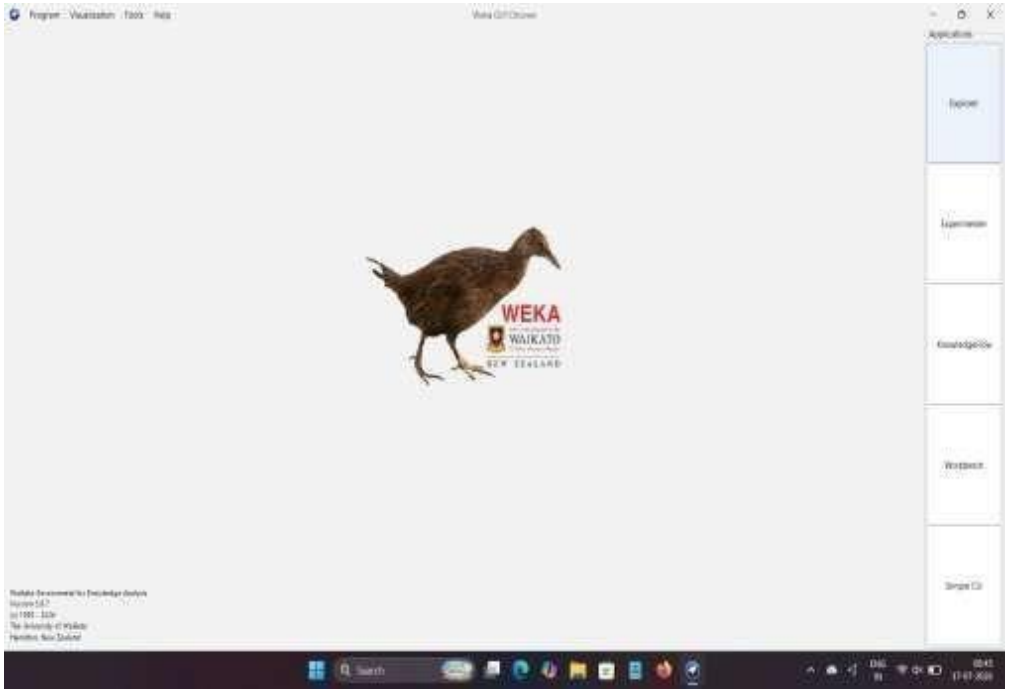
Procedure:

Step 1: Open WEKA

• Open WEKA.

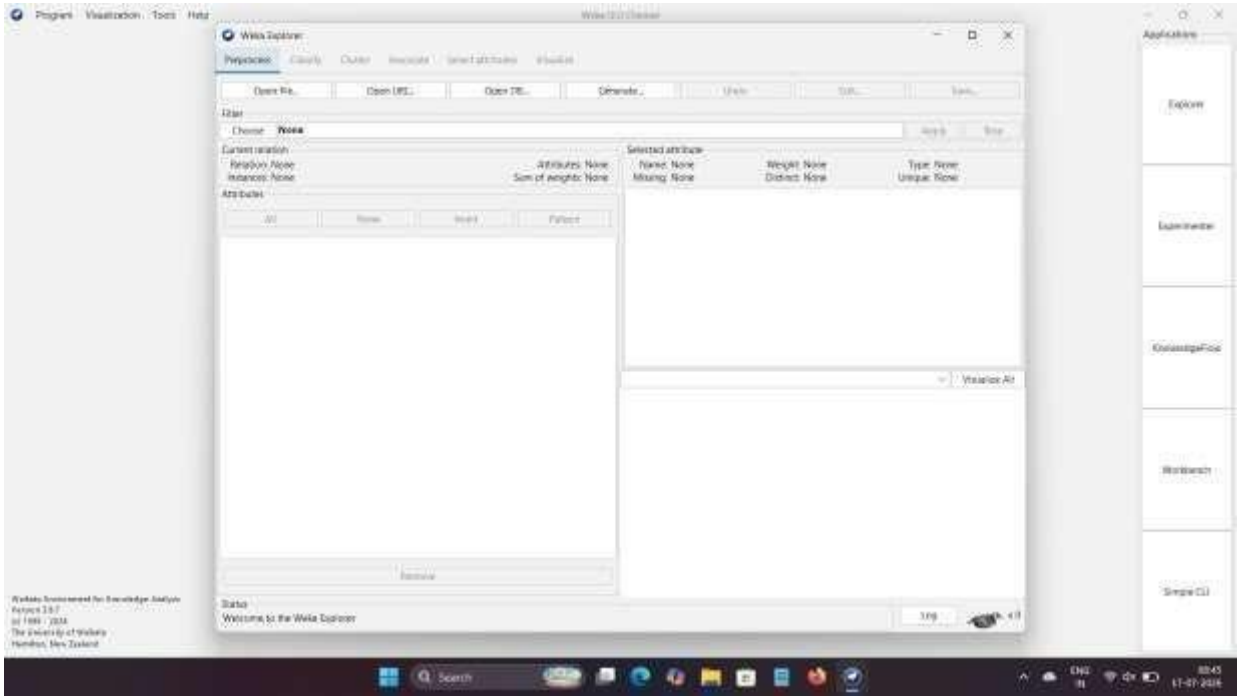
• The GUI Chooser window appears.

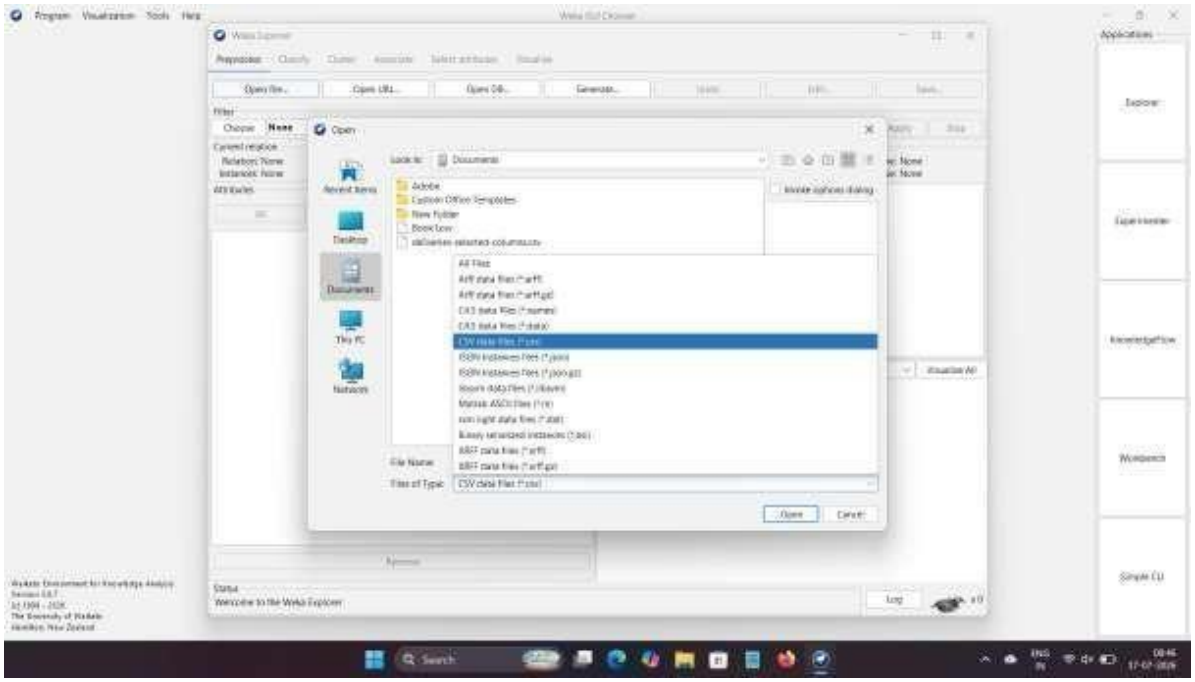
• Click Explorer.



Step 2: Load the CSV File

- In the Preprocess tab, click Open file.
- Browse and select your CSV (.csv) file.
- Click Open.

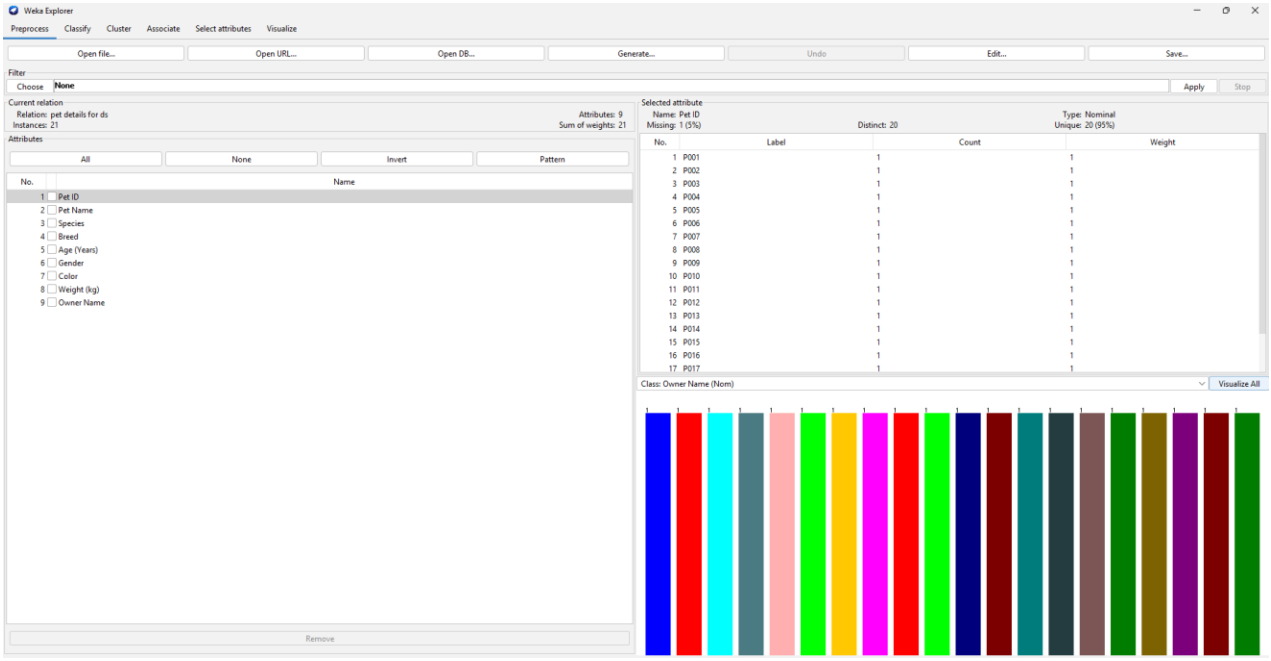


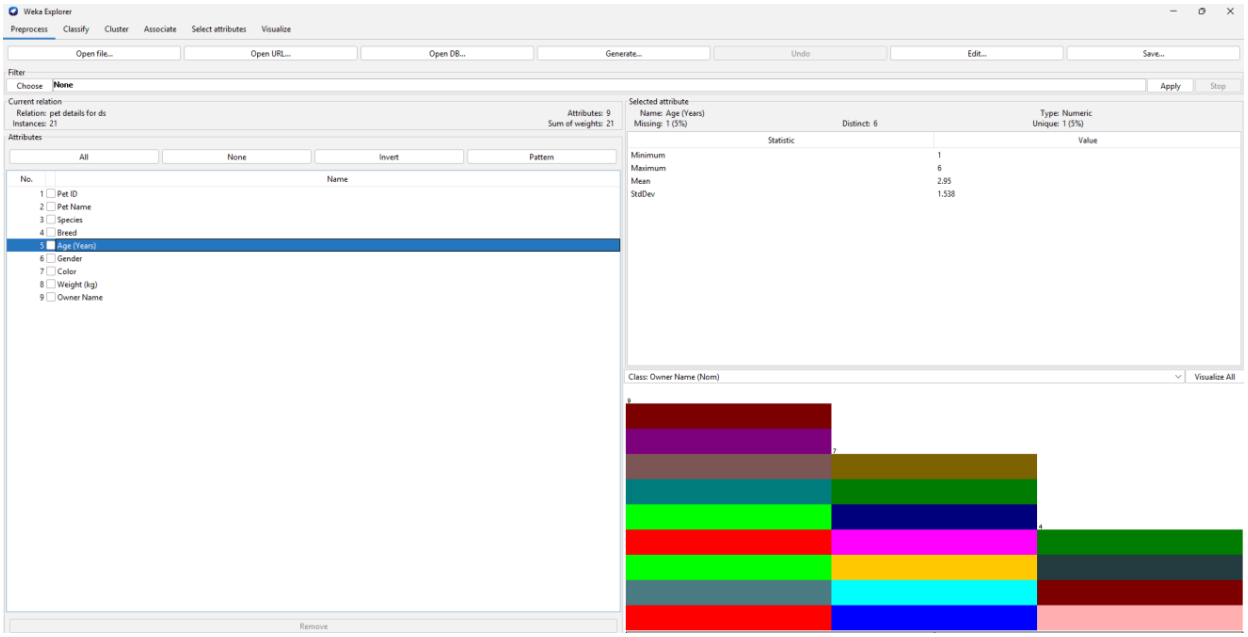


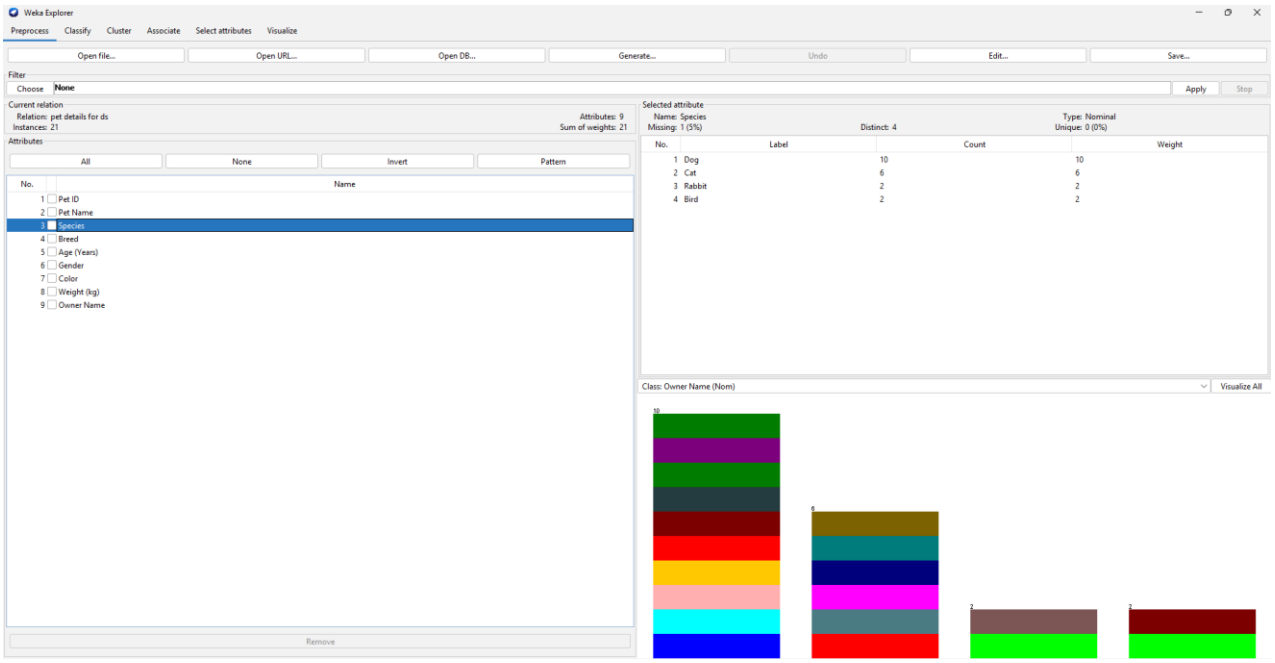
Step 3: Interpretation

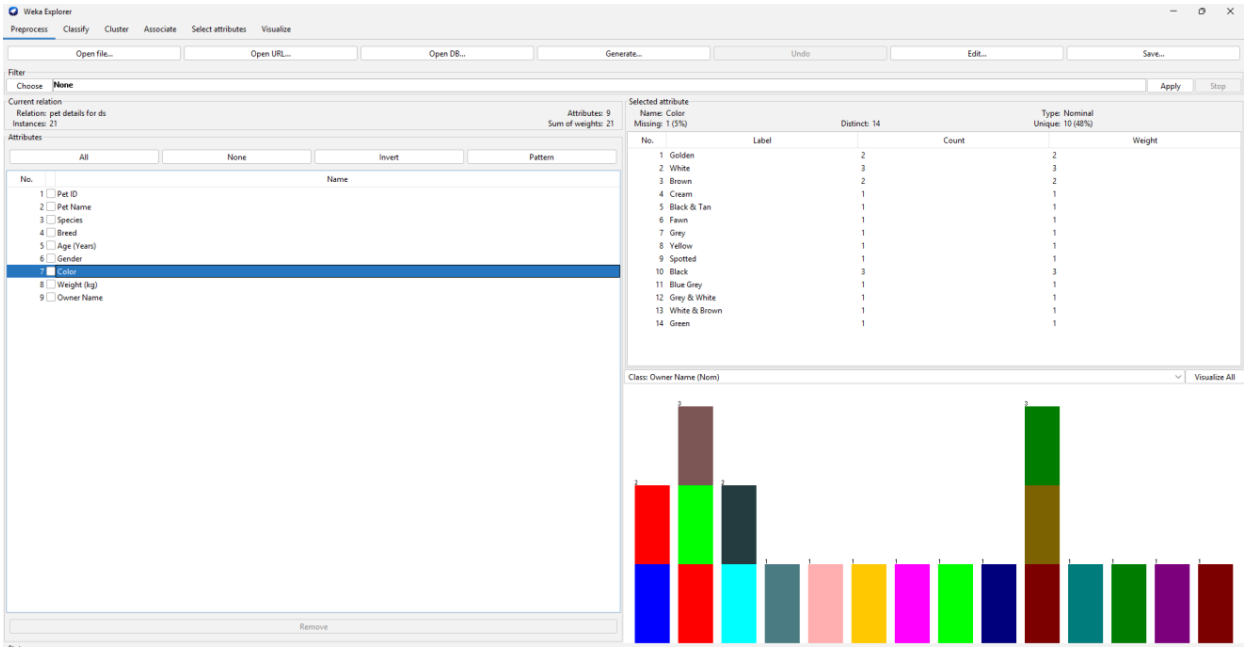
• Select any attribute (such as Team1, Team2, Winner and Venue) from the attribute list.

• WEKA displays the attribute statistics and generates a graph showing the data distribution for the selected attribute



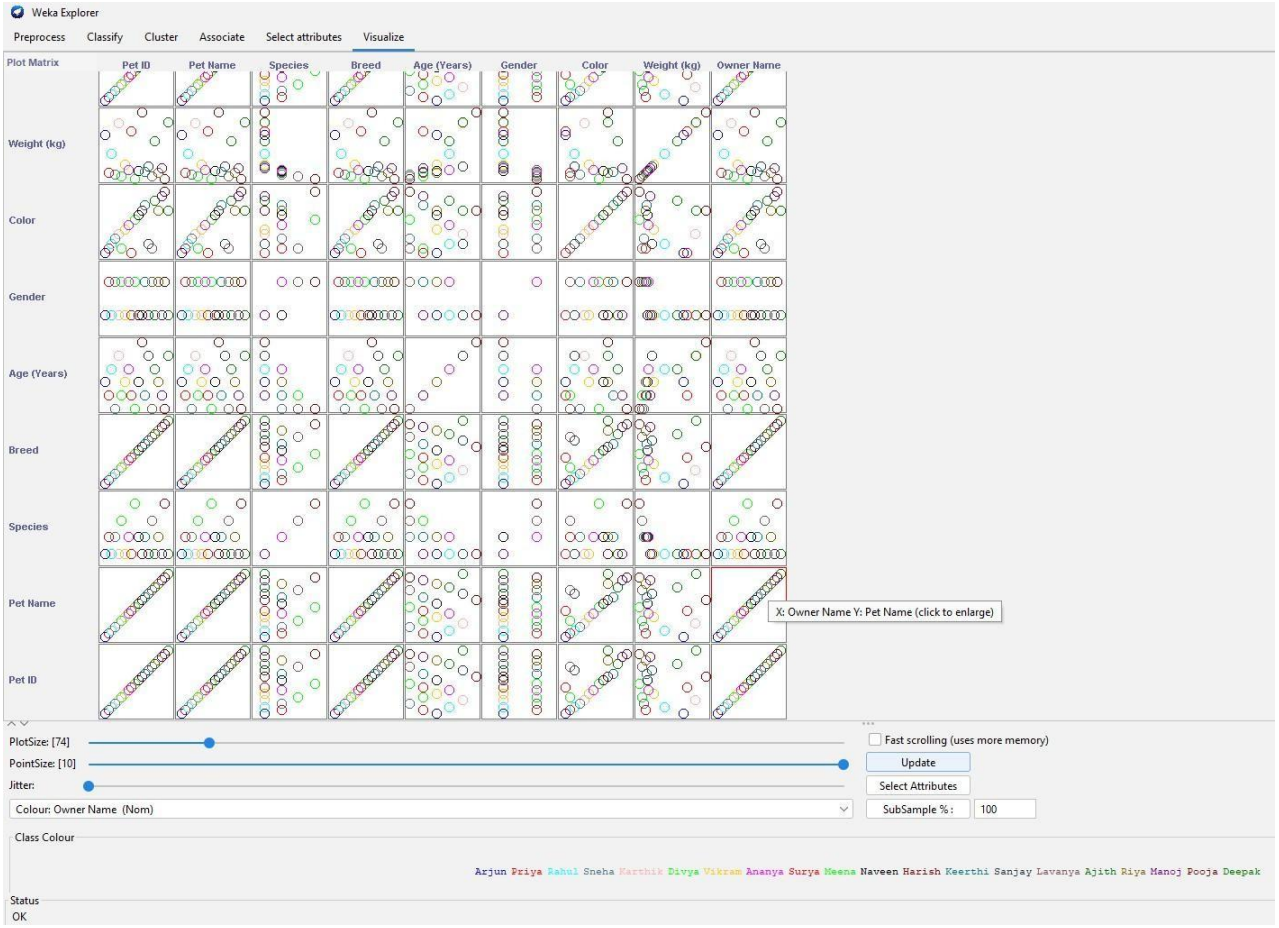



Weka Explorer
Preprocess Classify Cluster Associate Select attributes Visualize
Open file... Open URL... Open DB... Generate... Undo Edit... Save...
Filter
Choose None
Apply Stop
Current relation
Relation: pet details for ds
Instances: 21
Attributes: 9
Sum of weights: 21
Attributes
All None Invert Pattern
No. Name
1 Pet ID
2 Pet Name
3 Species
4 Breed
5 Age (Years)
6 Gender
7 Color
8 Weight (kg)
9 Owner Name
Remove
Selected attribute
Name: Species
Missing: 1 (5%)
Distinct: 4
Type: Nominal
Unique: 0 (0%)
No. Label Count Weight
1 Dog 10 10
2 Cat 6 6
3 Rabbit 2 2
4 Bird 2 2
Class: Owner Name (Nom)
Visualize All


Weka Explorer
Preprocess Classify Cluster Associate Select attributes Visualize
Open file... Open URL... Open DB... Generate... Undo Edit... Save...
Filter
Choose None
Apply Stop
Current relation
Relation: pet details for ds
Instances: 21
Attributes: 9
Sum of weights: 21
Attributes
All None Invert Pattern
No. Name
1 Pet ID
2 Pet Name
3 Species
4 Breed
5 Age (Years)
6 Gender
7 Color
8 Weight (kg)
9 Owner Name
Remove
Selected attribute
Name: Color
Missing: 1 (5%)
Distinct: 14
Type: Nominal
Unique: 10 (48%)
No. Label Count Weight
1 Golden 2 2
2 White 3 3
3 Brown 2 2
4 Cream 1 1
5 Black & Tan 1 1
6 Fawn 1 1
7 Grey 1 1
8 Yellow 1 1
9 Spotted 1 1
10 Black 3 3
11 Blue Grey 1 1
12 Grey & White 1 1
13 White & Brown 1 1
14 Green 1 1
Class: Owner Name (Nom)
Visualize All

Step 4: Visualize

• Open Visualize
• Click the Visualize tab in WEKA Explorer.
• Select X-axis = Match ID, Y-axis = Runs, and Colour = Winner.
• View Scatter Plot
• WEKA generates a scatter plot showing the relationship between Match ID and Runs.
• The data points are color-coded based on the Winner, making it easy to compare the distribution.



Conclusion:

The process is by using the weka tool how we get the out for the csv file and get the visual graph representation was completed successfully..

Ex.No. 1C	Explore the data types and variables and analyze the features
DATE:31.07.2026	

Aim:

To explore the data types and variables and analyze the features using Weka data mining tool.

Procedure:

- Open Weka Open Weka
- Launch the Weka application on your computer.
- Load the CSV File Load the CSV File
- In the Weka GUI, click on "Explorer".
- Click on the "Open file..." button.
- Navigate to the location where you saved the CSV file, select it, and click "Open".
- Explore Data Types and Variables Preprocess Tab
- Once the file is loaded, you will be in the "Preprocess" tab.
- The "Attributes" panel will list all the variables (columns) in your dataset. Data Types
- Each attribute will have a data type such as numeric, nominal (categorical), or string.
- Click on each attribute to see its details, including the data type and distribution of values.
- Analyze Features

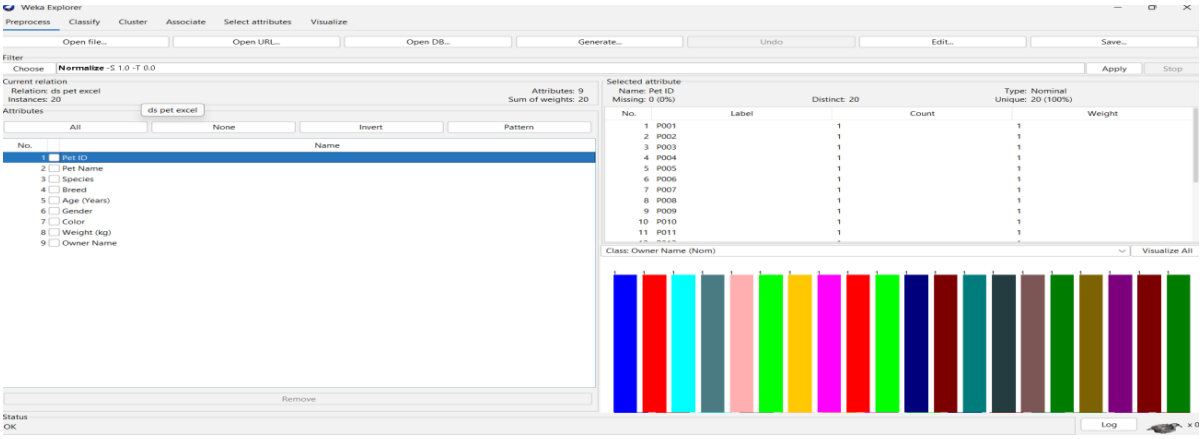
a. Summary Statistics

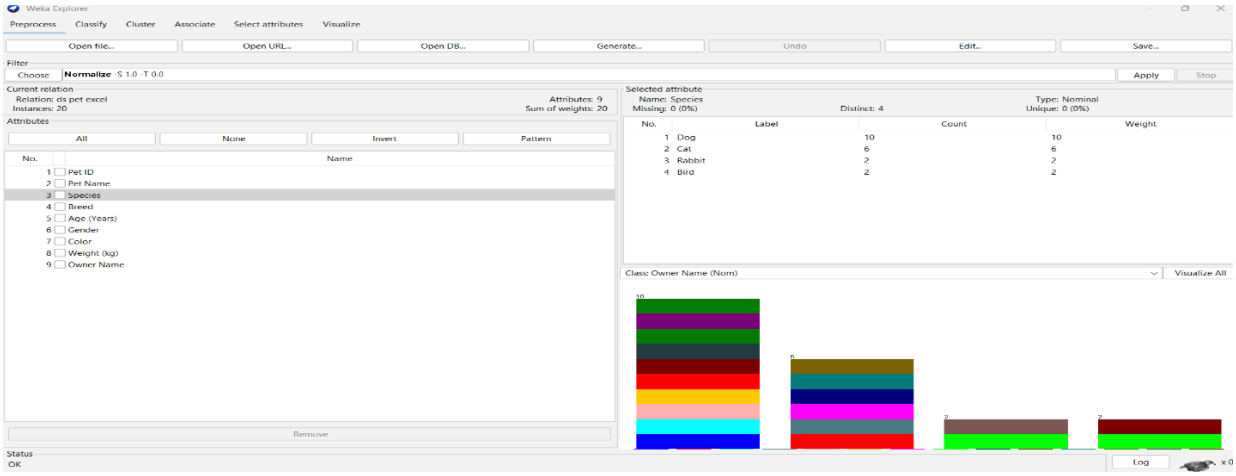
- For numeric attributes, you can view summary statistics like mean, min, max, and standard deviation.

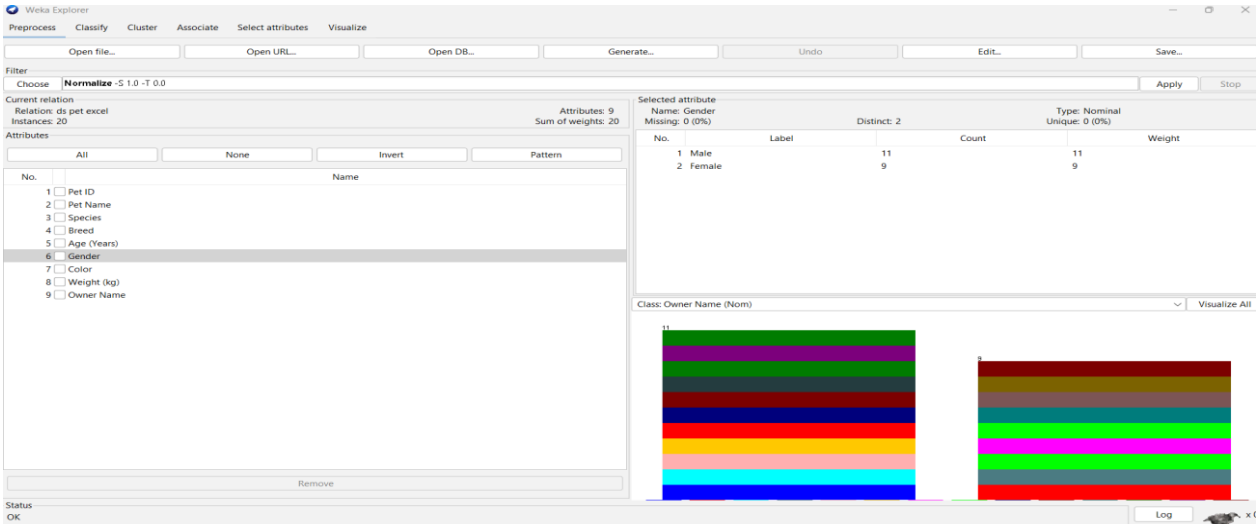
For nominal attributes, you can see the frequency of each category b.Correlation Analysis

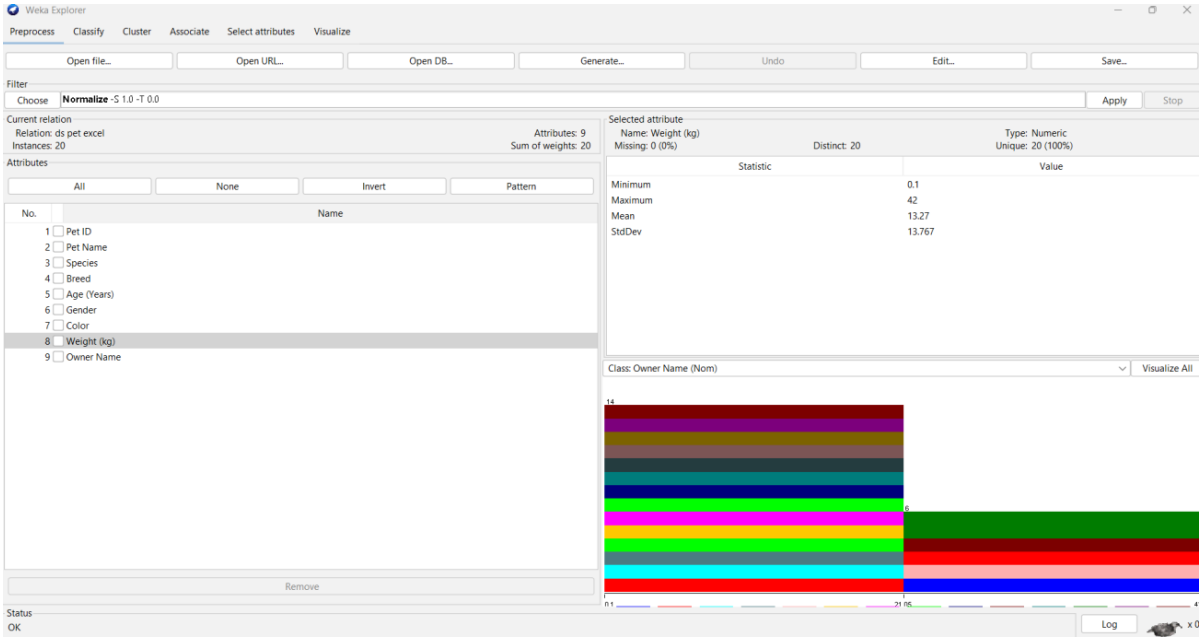
- Click on the "Select attributes" tab.
- Use the "Correlation-based Feature Selection" method to evaluate the relationship between features.
- Click "Start" to run the analysis. visual Exploration

- Click on the "Visualize" tab to create scatter plots, histograms, or other visualizations.
- Select different pairs of attributes to visualize relationships and distributions.









Selected attribute

Name: Weight (kg) Type: Numeric

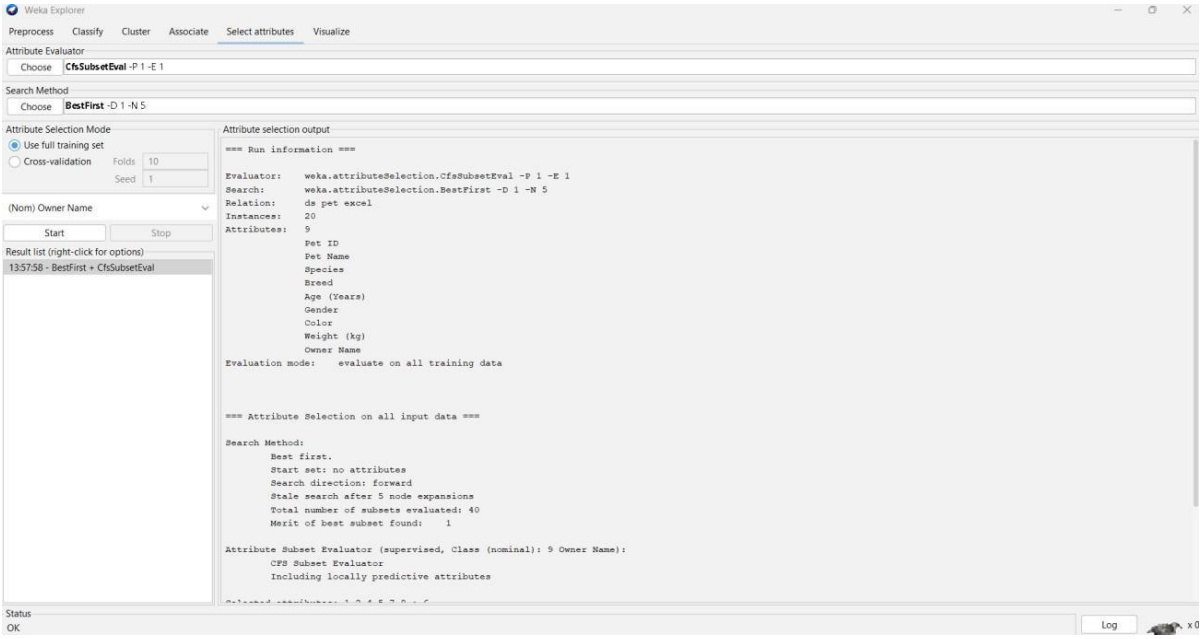
Missing: 0 (0%) Distinct: 20 Unique: 20 (100%)

Statistic	Value
Minimum	0.1
Maximum	42
Mean	13.27
StdDev	13.767

Selected attribute

Name: Species | Type: Nominal
Missing: 0 (0%) | Distinct: 4 | Unique: 0 (0%)

No.	Label	Count	Weight
1	Dog	10	10
2	Cat	6	6
3	Rabbit	2	2
4	Bird	2	2



Result:

Thus the dataset was thoroughly explored for data types and variables. Summary statistics and frequency distributions provided an understanding of each attribute's characteristics

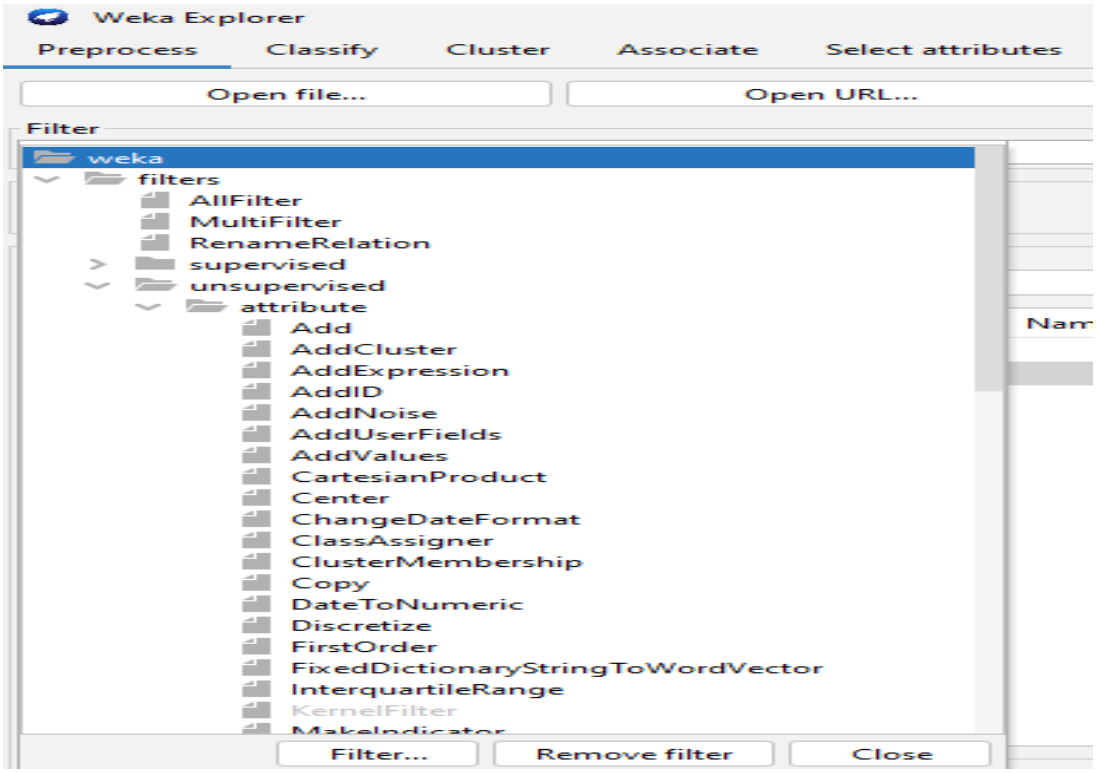
Ex.No. 1D	Normalize the datasets using preprocessing techniques using WEKA tool
DATE:31.07.2026	

Aim:

To Normalize the datasets using preprocessing techniques using WEKA tool

Procedure:

1. Open Weka
   - Open Weka
   - Launch the Weka application on your computer.
2. Load the CSV File
   - Load the CSV File
   - In the Weka GUI, click on "Explorer".
   - Click on the "Open file..." button.
   - Navigate to the location where you saved the CSV file, select it, and click "Open".



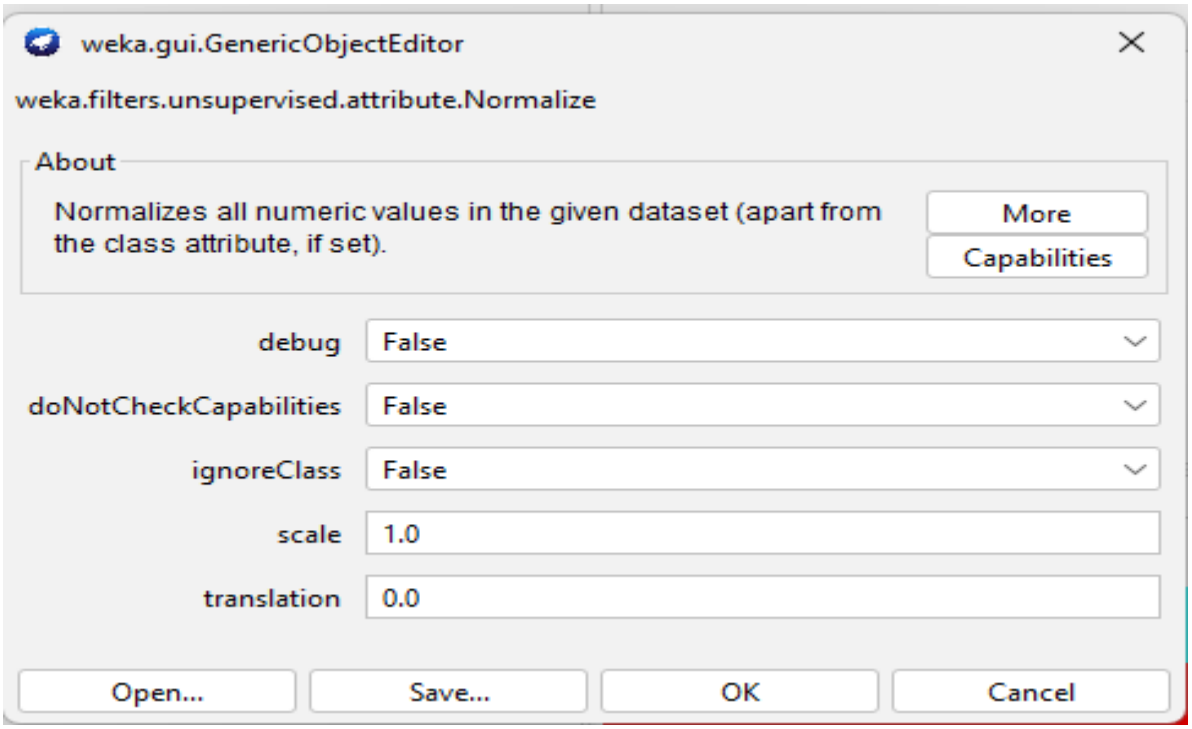
3. Apply Normalization
   - Go to the Preprocess Tab

- Ensure that your dataset is loaded and visible in the "Preprocess" tab.
- Click on the "Choose" button under the "Filter" section.

- Navigate to "unsupervised" > "attribute" > "Normalize".

3. Configure Normalize Filter

- Once the Normalize filter is selected, click on the "Normalize" filter to open its configuration settings.
- Set the desired range for normalization (e.g., default [0, 1]).



4. Apply the Filter

- Click the "Apply" button to normalize the dataset.
- The dataset should now reflect the normalized values.

5. Save the Normalized Dataset

- Save the Normalized Data
- Click on "Save" to save the normalized dataset.
- Choose the location and name for the new CSV file, and ensure the file type is set to CSV.

Result:

By using Weka's Normalize filter, the numerical attributes in the dataset were scaled to a standard range, ensuring better performance for subsequent data mining and analysis processes.

Ex.No. 2	Install and explore the features of NumPy, SciPy, Jupyter, Statsmodels and Pandas packages.
DATE:31.07.26	

Aim:

To install and explore the features of NumPy, SciPy, Jupyter, Statsmodels, and Pandas packages.

PROCEDURE

Step 1: Download and Install Python with IDLE Download Python:

Go to the official Python website: https://www.python.org/. Click on the "Downloads" tab.

Download the latest version of Python for your operating system. Install Python:

Run the downloaded installer.

During installation, make sure to check the box that says "Add Python to PATH" before clicking "Install Now".

Follow the prompts to complete the installation. Step 2: Install Necessary Packages

Open Command Prompt or Terminal:

For Windows: Press Win + R, type cmd, and press Enter.

In the command prompt or terminal, type the following commands to install the required packages:

pip install numpy scipy statsmodels pandas matplotlib

Press Enter after each command and wait for the installation to complete. Step 3: Open Python IDLE

Open Python IDLE:

For Windows: Find Python IDLE in the start menu or search for "IDLE". Step 4: Write and Run the Python Script

Create a New Script:

In Python IDLE, go to File > New File to open a new script window.

Step 5: Explore NumPy Features:

Import the NumPy library and create NumPy arrays and perform basic operations like addition, mean calculation, and standard deviation.

Step 6: Explore Pandas Features:

Import the Pandas library and create a DataFrame with sample data and perform DataFrame operations such as calculating mean, median, and mode.

Step 7:Explore SciPy Features:

Import the SciPy stats module and perform a t-test on two sets of data and print the t-statistic and p-value to interpret the results of the test.

Step 8:Explore Statsmodels Features:

Import the Statsmodels library and fit a simple linear regression model and use sample data to predict a dependent variable based on independent variables.

Step 9:Plotting with Matplotlib in Jupyter Notebook:

Import the Matplotlib library to create plots and plot a scatter plot to visualize the relationship between two variables.

Program:

Import necessary libraries import numpy as np

import pandas as pd import scipy.stats as stats

import statsmodels.api as sm import matplotlib.pyplot as plt

1. NumPy: Numerical Python

Create a NumPy array and perform basic operations

a = np.array([1, 2, 3, 4, 5])

b = np.array([10, 20, 30, 40, 50])

Basic operations sum_ab = a + b mean_a = np.mean(a)

std_b = np.std(b)

print("NumPy Operations:") print("Sum of arrays a and b:", sum_ab) print("Mean of array a:", mean_a)

```
print("Standard deviation of array b:", std_b) print("\n")

# 2. Pandas: Data manipulation and analysis # Create a Pandas DataFrame

data = {

'Age': [23, 25, 28, 22, 30],

'Weight': [70, 65, 78, 55, 72],

'Height': [165, 170, 175, 160, 168]

}

df = pd.DataFrame(data) # DataFrame operations

mean_age = df['Age'].mean() median_weight = df['Weight'].median()

mode_height = stats.mode(df['Height'], keepdims=True)[0][0] print("Pandas Operations:")

print("Mean age:", mean_age) print("Median weight:", median_weight) print("Mode height:", mode_height) print("\n")

# 3. SciPy: Scientific and technical computing # Perform a t-test on two sets of data

t_stat, p_value = stats.ttest_ind(a, b) print("SciPy Operations:")

print("T-statistic:", t_stat) print("P-value:", p_value) print("\n")

# 4. Statsmodels: Statistical models

# Fit a simple linear regression model

X = sm.add_constant(df[['Age', 'Weight']]) # Add constant term y = df['Height']

model = sm.OLS(y, X) results = model.fit()

print("Statsmodels OLS Regression Results:") print(results.summary())

print("\n")

# 5. Matplotlib: Plotting graphs

# Plot a graph

plt.scatter(df['Age'], df['Height'], color='blue') plt.xlabel('Age')

plt.ylabel('Height') plt.title('Age vs. Height') plt.show()
```

Output:

NumPy Operations:

Sum of arrays a and b: [11 22 33 44 55] Mean of array a: 3.0

Standard deviation of array b: 14.142135623730951

Pandas Operations:

Mean age: 25.6

Median weight: 70.0

Mode height: 160

SciPy Operations:

T-statistic: -3.7994267415435763

P-value: 0.005240940516978861

Statsmodels OLS Regression Results:

OLS Regression Results

==========

Dep. Variable: Height R-squared:

0.717

Model: OLS Adj. R-squared:

0.434

Method: Least Squares F-statistic:

2.535

Date: Sun, 21 Jul 2024 Prob (F-statistic):

0.283

Time: 15:11:09 Log-Likelihood:

-11.989

No. Observations: 5 AIC:

29.98

Df Residuals: 2 BIC:

28.81

Df Model: 2

Covariance Type: nonrobust

==========

coef std err t P>|t| [0.025 0.975]

const 128.8596 17.631 7.308 0.018 52.997 204.722

Age

0.2859

0.919

0.311

0.785

-3.670

4.241

Weight

0.4621

0.358

1.291

0.326

-1.078

2.003

==========

Omnibus: nan Durbin-Watson: 1.709

Prob(Omnibus):

nan Jarque-Bera (JB):

0.547

Skew:

0.315 Prob(JB):

0.761

Kurtosis:

1.507 Cond. No.

685.

==========

Result:

We explore the features of NumPy, SciPy, Pandas, Statsmodels, and Matplotlib in Python IDLE successfully.

Ex.No. 3	Working with Numpy Arrays
DATE:06.08.2026	

Create an array using “np” library with different methods like arrange(), random.rand(), empty()

Create a Numpy array filled with zeros and ones.

Create Student Database with five columns and remove the columns in Numpy arrary that contains non-numeric value.

Get the row numbers of Numpy array having marks greater than 60 from the student database

Aim:

To create and manipulate Numpy arrays using various methods, manage a student database, and perform operations to analyze the data.

Procedure

Create Arrays Using Different Methods:

Use np.arange() to create an array with a range of values.

Use np.random.rand() to create an array with random values.

Use np.empty() to create an array with uninitialized values.

Create Numpy Arrays Filled with Zeros and Ones:

Use np.zeros() to create an array filled with zeros.

Use np.ones() to create an array filled with ones.

Create and Manipulate a Student Database:

Create a Numpy array representing a student database with columns such as Name, Age, Marks, etc.

Remove columns with non-numeric values.

Analyze the Student Database:

Identify and retrieve the rows where the marks are greater than 60.

Program 3.a

```
import numpy as np

# Create an array from 0 to 8

array_arange = np.arange(8)

print("Array using arange():", array_arange) # Create a 3x3 array with random values
array_random = np.random.rand(3, 3)

print("Array using random.rand():\n", array_random)

# Create a 2x2 empty array

array_empty = np.empty((2, 2))

print("Array using empty():\n", array_empty)
```

3.b

```
# Create a 3x3 array filled with zeros

array_zeros = np.zeros((3, 3))

print("Array filled with zeros:\n", array_zeros) # Create a 3x3 array filled with ones
array_ones = np.ones((3, 3))

print("Array filled with ones:\n", array_ones)
```

3.c

```
# Create a structured array for the student database

student_dtype = np.dtype([('ID', 'i4'), ('Name', 'U10'), ('Age', 'i4'), ('Marks', 'f4'), ('Grade',
'U2')])

students = np.array([

(1, 'Alice', 10, 55.0, 'B'),

(2, 'Bob', 28, 85.0, 'A'),

(3, 'Charlie', 32, 80.0, 'D'),

(4, 'David', 24, 90.0, 'E'), (5, 'Eve', 21, 50.0, 'F')

], dtype=student_dtype) print("Student Database:\n", students) # Extract only the
numeric columns
```

numeric_students = students[['ID', 'Age', 'Marks']] print("Numeric Student Database:\n", numeric_students)

3.d

Get row numbers where Marks are greater than 60 rows_with_high_marks = np.where(numeric_students['Marks'] > 60)[0] print("Row numbers with Marks > 60:", rows_with_high_marks

Output:

3.a

Array using arange(): [0 1 2 3 4 5 6 7] Array using random.rand():

[[0.78634636 0.52522964 0.24912684]

[0.13272032 0.58524295 0.80145625]

[0.9957435 0.69236683 0.23788024]]

Array using empty():

[[8.46096606e-312 8.46300540e-312] [8.46096626e-312 8.46096618e-312]]

3.b

Array filled with zeros:

[[0. 0. 0.]

[0. 0. 0.]

[0. 0. 0.]]

Array filled with ones:

[[1. 1. 1.]

[1. 1. 1.]

[1. 1. 1.]]

3.c

Student Database:

[(1, 'Alice', 10, 55., 'B') (2, 'Bob', 28, 85., 'A')

(3, 'Charlie', 32, 80., 'D') (4, 'David', 24, 90., 'E')

(5, 'Eve', 21, 50., 'F')]

Numeric Student Database:

[(1, 10, 55.) (2, 28, 85.) (3, 32, 80.) (4, 24, 90.) (5, 21, 50.)]

3.d

Row numbers with Marks > 60: [0 2 3]

Result:

Working with numpy arrays has been successfully executed.

Ex.No. 4	Working with Pandas Data Frames.
DATE:04.09.26	

Objective

To work with Pandas DataFrames and NumPy arrays by:

Loading a CSV file as a Pandas DataFrame and obtaining basic information about the data.

Finding the maximum and minimum elements in a NumPy array.

Adding rows and columns to a NumPy array.

Adding, removing, and updating rows and columns using Pandas.

Procedure

Part A: Loading CSV File as a Pandas DataFrame

Specify the path of the CSV file.

Use pd.read_csv() to load the CSV file into a Pandas DataFrame.

Obtain basic information about the DataFrame using info().

Display the first five rows using head().

Obtain the statistical summary using describe().

Part B: Finding Maximum and Minimum Elements in a NumPy Array

Create a NumPy array.

Use np.max() to find the maximum element.

Use np.min() to find the minimum element.

Display the maximum and minimum elements.

Part C: Adding Rows and Columns in a NumPy Array

Create a two-dimensional NumPy array using reshape().

Create a new row.

Add the new row using np.vstack().

Create a new column.

Add the new column using np.hstack().

Display the modified arrays.

Part D: Adding, Removing and Updating Rows/Columns Using Pandas

Create a new row as a Pandas DataFrame.

Add the row to the existing DataFrame using pd.concat().

Remove a column using drop().

Add a new column using DataFrame column assignment.

Update a specific cell using loc[].

Display the modified DataFrame after each operation.

Program

```
import pandas as pd import numpy as np
# ------------------------------------------------
# Part A: Loading CSV file as a Pandas DataFrame # ------------------------------------------------
csv_file_path = ( r"D:\KRCT\IT\26-27\DS\LAB\exp 4\Iris.csv") df = pd.read_csv(
csv_file_path, header=None, names=[
'sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'species'
    ]
)
# Display basic information
print("Basic Information about the
DataFrame:") print(df.info())
print("\nFirst 5 rows of the DataFrame:")
print(df.head())

print("\nStatistical Summary of the
DataFrame:") print(df.describe())
#  ---------------------------------
# Part B: Finding Maximum and Minimum
Elements # ------------------------------------------
```

```
------
arr = np.array(
    [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
)
max_element =
np.max(arr) min_element
= np.min(arr) print(
    f"\nMaximum element in the array: {max_element}"
)

print(
    f"Minimum element in the array: {min_element}"
)
#  ---------------------------------
# Part C: Adding Rows and Columns in NumPy
Array # ------------------------------------------------
# Reshape array into 2D
array arr_2d =
arr.reshape(-1, 1)
# Add a new row
new_row =
np.array([
    [7],
    [8],
    [9]
])
arr_with_new_row = np.vstack([
    arr_2d,
    new_row
```

```
])
print(
    "\nNumPy array with new row added:\n",
    arr_with_new_row
)
# Add a new
column
new_column =
np.array([
    0, 1, 1, 2, 3, 5, 8,
    13, 21, 34, 55, 89, 144, 233
]).reshape(-1, 1)
arr_with_new_column =
np.hstack([
    arr_with_new_row[:new_column.shape[0]],
    new_column
])
print(
    "\nNumPy array with new column
    added:\n", arr_with_new_column
)
# ---------------------------------
# Part D: Adding, Removing and
Updating # Using Pandas
# ---------------------------------
# Adding a new row
new_row_df =
pd.DataFrame({
    'sepal_length': [5.5],
```

```
    'sepal_width': [3.5],
    'petal_length': [1.3],
    'petal_width': [0.2],
    'species': ['Iris-setosa']
})
df = pd.concat(
    [df, new_row_df],
    ignore_index=Tru
    e
)
print(
    "\nDataFrame after adding a new
    row:\n", df.tail()
)


# Removing a
column df =
df.drop(
    columns=['sepal_width']
)
print(
    "\nDataFrame after removing a column:\n",
    df.head()
)
# Adding a new column df['new_column'] = np.arange(len(df)) print(
"\nDataFrame after adding a new column:\n", df.head()
)
# Updating a specific cell df.loc[0, 'sepal_length'] = 999 print(
"\nDataFrame after updating a specific cell:\n", df.head()
```

)

Output

Basic Information about the DataFrame:

<class 'pandas.core.frame.DataFrame'> Index: 151 entries, Id to 150
Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	sepal_length	151 non-null	object
1	sepal_width	151 non-null	object
2	petal_length	151 non-null	object
3	petal_width	151 non-null	object
4	species	151 non-null	

object dtypes:

object(5)

memory usage:

7.1+ KB None

First 5 rows of the DataFrame:

	sepal_length	sepal_width	petal_length	petal_width	species
Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
1	5.1	3.5	1.4	0.2	Iris-setosa
2	4.9	3.0	1.4	0.2	Iris-setosa
3	4.7	3.2	1.3	0.2	Iris-setosa
4	4.6	3.1	1.5	0.2	Iris-setosa

Statistical Summary of the DataFrame:

	sepal_length	sepal_width	petal_length	petal_width	species
count	151	151	151	151	151
unique	36	24	44	23	4

top	5.0	3.0	1.5	0.2	Iris-setosa
freq	10	26	14	28	50

Maximum element in the array:

9 Minimum element in the

array: 1

NumPy array with new row

added: [[3]

[1]

[4]

[1]

[5]

[9]

[2]

[6]

[5]

[3]

[5]

[7]

[8]

[9]]

NumPy array with new column

added: [[3 0]

[1 1]

[4 1]

[1 2]

[5 3]

[9 5]

[2 8]

[6 13]

[5 21]

[3 34]

[5 55]

[7 89]

[8 144]

[9 233]]

DataFrame after adding a new row:

	sepal_length	sepal_width	petal_length	petal_width	species
147	6.3	2.5	5.0	1.9	Iris-virginica
148	6.5	3.0	5.2	2.0	Iris-virginica
149	6.2	3.4	5.4	2.3	Iris-virginica
150	5.9	3.0	5.1	1.8	Iris-virginica
151	5.5	3.5	1.3	0.2	Iris-setosa

DataFrame after removing a column:

	sepal_length	petal_length	petal_width	species
0	SepalLengthCm	PetalLengthCm	PetalWidthCm	Species
1	5.1	1.4	0.2	Iris-setosa
2	4.9	1.4	0.2	Iris-setosa
3	4.7	1.3	0.2	Iris-setosa
4	4.6	1.5	0.2	Iris-setosa

DataFrame after adding a new column:

	sepal_length	petal_length	petal_width	species	new_column
0	SepalLengthCm	PetalLengthCm	PetalWidthCm	Species	0
1	5.1	1.4	0.2	Iris-setosa	1
2	4.9	1.4	0.2	Iris-setosa	2

3	4.7	1.3	0.2	Iris-setosa	3
4	4.6	1.5	0.2	Iris-setosa	4

DataFrame after updating a specific cell:

	sepal_length	petal_length	petal_width	species	new_column
0	999	PetalLengthCm	PetalWidthCm	Species	0
1	5.1	1.4	0.2	Iris-setosa	1
2	4.9	1.4	0.2	Iris-setosa	2
3	4.7	1.3	0.2	Iris-setosa	3
4	4.6	1.5	0.2	Iris-setosa	4

Conclusion

Thus, the CSV file was successfully loaded into a Pandas DataFrame and its basic information and statistical summary were obtained. The maximum and minimum elements of a NumPy array were identified, and rows and columns were added to the NumPy array. The Pandas DataFrame was also successfully modified by adding, removing, and updating rows and columns.

Ex.No. 5	Descriptive Analysis: Reading Data from Different Files and Web Sources Using Python
DATE: 18.09.2026	

Objective

To perform descriptive analysis by reading data from different files and web sources using Python. This includes:

- Creating employee data and reading it as a CSV file.
- Analyzing the correlation between the features of the Iris dataset.
- Analyzing the correlation between the features of the Diabetes dataset.
- Generating histograms for the Iris and Diabetes datasets.

Procedure

Part A: Creating Employee Data and Reading as CSV

1. Create employee data containing employee name, salary, and years of experience.
2. Create a Pandas DataFrame using the employee data.
3. Save the DataFrame as a CSV file.
4. Read the CSV file using pandas.read_csv().
5. Display the employee DataFrame.

Part B: Iris Dataset and Correlation Analysis

1. Load the Iris dataset using seaborn.
2. Display the first few records of the dataset.
3. Calculate the correlation matrix for the numerical features.
4. Display the correlation matrix.

Part C: Diabetes Dataset and Correlation Analysis

1. Download the Diabetes dataset from Kaggle.

2. Load the dataset using pandas.read_csv().
3. Display the first few records of the dataset.
4. Calculate the correlation matrix for the numerical features.
5. Display the correlation matrix.

Part D: Generating Histograms

1. Generate histograms for the Iris dataset.

1. Display the histograms using Matplotlib.
2. Generate histograms for the Diabetes dataset.
3. Display the histograms using Matplotlib.

Program

```
import pandas as pd import numpy as np
import matplotlib.pyplot as plt import seaborn as sns
# ------------------------------------------------
# Part A: Create employee details and read as CSV # ------------------------------------------------
employee_data = { 'EmployeeName': [
'John Doe', 'Jane Smith', 'Emily Davis', 'Michael Brown', 'Jessica Wilson'
],
'Salary': [ 70000,
80000,
75000,
82000,
78000
],
'YearsOfExperience': [ 5,
7,
6,
8,
7
```

```
]
}
employee_df = pd.DataFrame(employee_data) # Save as CSV
employee_df.to_csv( 'employee_data.csv', index=False
)
# Read the CSV file employee_df = pd.read_csv(
'employee_data.csv'
)
print("Employee DataFrame:") print(employee_df)
# ------------------------------------------------
# Part B: Iris Dataset and Correlation Analysis # ------------------------------------------------
iris_df = sns.load_dataset('iris') print("\nIris DataFrame:") print(iris_df.head())
# Correlation matrix iris_corr = iris_df.corr(
numeric_only=True
)
print("\nCorrelation Matrix for Iris Dataset:") print(iris_corr)
# ------------------------------------------------
# Part C: Diabetes Dataset and Correlation Analysis
# ------------------------------------------------
# Replace the path with the actual location # of the Diabetes CSV file
diabetes_df = pd.read_csv( "C:\\Users\\Renuka\\Desktop\\python programs\\diabetes.csv"
)
print("\nDiabetes DataFrame:") print(diabetes_df.head())
# Correlation matrix diabetes_corr = diabetes_df.corr(
numeric_only=True
)
print("\nCorrelation Matrix for Diabetes Dataset:") print(diabetes_corr)
# ------------------------------------------------
# Part D: Generate Histograms
#  ------------------------------------------------
```

```
# Histogram for Iris dataset iris_df.hist(figsize=(10, 8)) plt.suptitle(
'Histograms of Iris Dataset'
)
plt.show()
# Histogram for Diabetes dataset diabetes_df.hist(figsize=(10, 8)) plt.suptitle(
'Histograms of Diabetes Dataset'
)
plt.show()
```

Output:

```
Employee DataFrame:
    EmployeeName  Salary  YearsOfExperience
0       John Doe   70000                  5
1     Jane Smith   80000                  7
2    Emily Davis   75000                  6
3  Michael Brown   82000                  8
4  Jessica Wilson   78000                  7


Iris DataFrame:
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa


Correlation Matrix for Iris Dataset:
              sepal_length  sepal_width  petal_length  petal_width
sepal_length      1.000000    -0.117570      0.871754     0.817941
sepal_width      -0.117570     1.000000     -0.428440    -0.366126
```

petal_length 0.871754 -0.428440 1.000000 0.962865

petal_width 0.817941 -0.366126 0.962865 1.000000

Diabetes DataFrame:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI
0	6	148	72	35	0	33.6
1	1	85	66	29	0	26.6
2	8	183	64	0	0	23.3
3	1	89	66	23	94	28.1
4	0	137	40	35	168	43.1

	DiabetesPedigreeFunction	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1
3	0.167	21	0
4	2.288	33	1

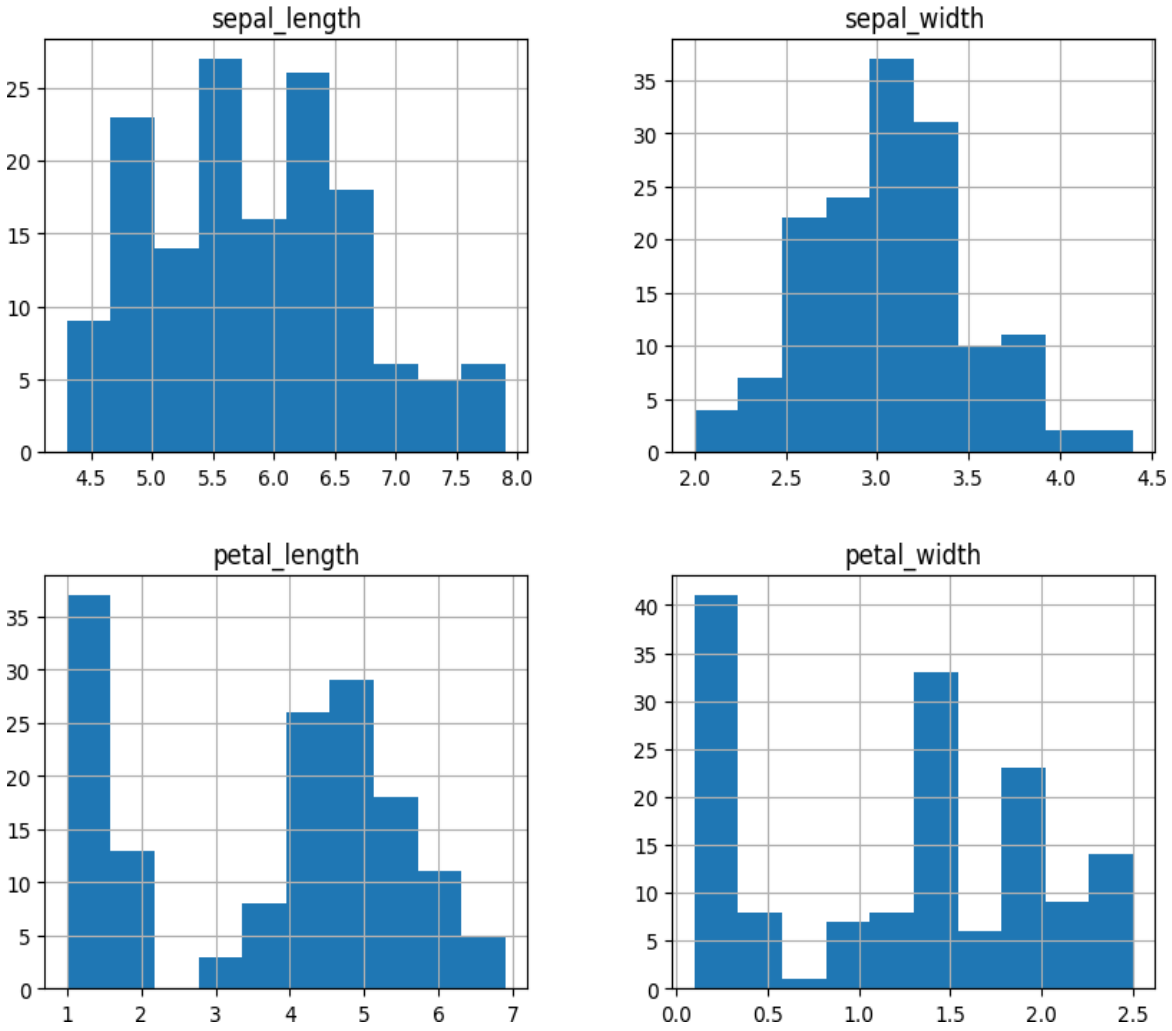
Correlation Matrix for Diabetes Dataset:

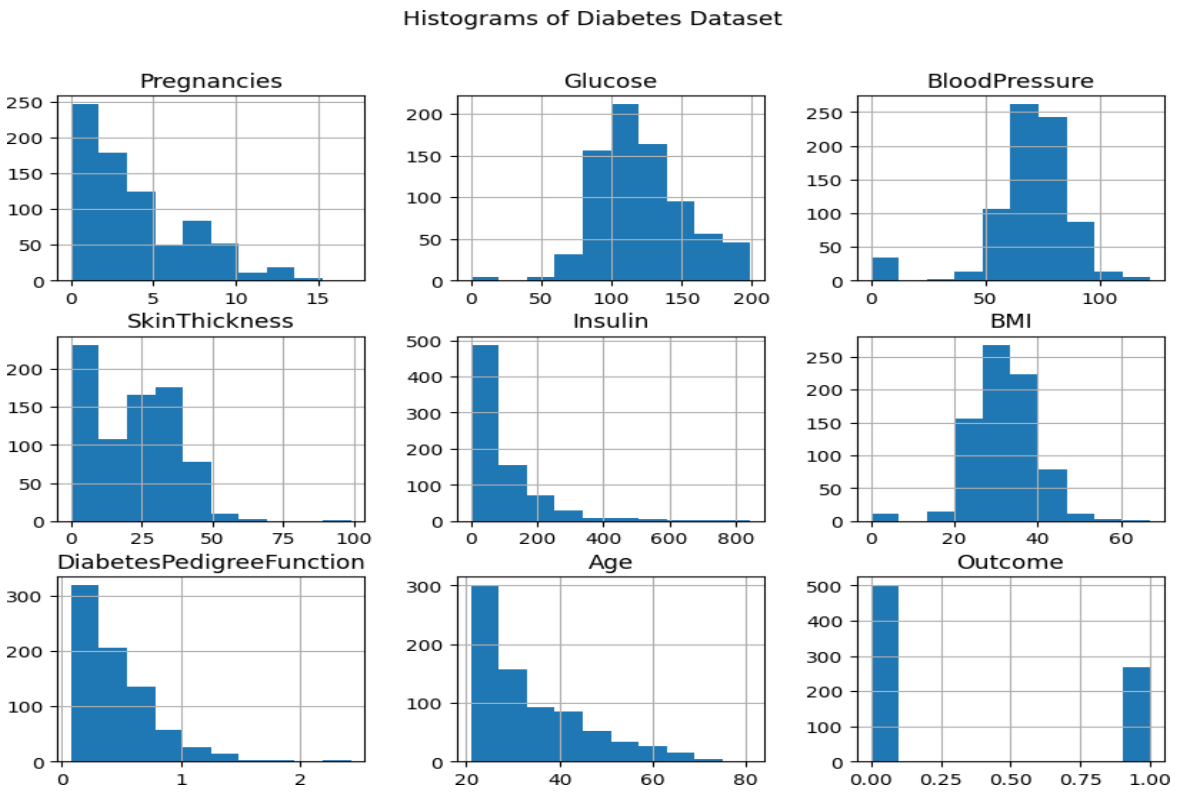
	Pregnancies	Glucose	BloodPressure	SkinThickness
Pregnancies	1.000000	0.129459	0.141282	-0.081672
Glucose	0.129459	1.000000	0.152590	0.057328
BloodPressure	0.141282	0.152590	1.000000	0.207371
SkinThickness	-0.081672	0.057328	0.207371	1.000000
Insulin	-0.073535	0.331357	0.088933	0.436783
BMI	0.017683	0.221071	0.281805	0.392573
DiabetesPedigreeFunction	-0.033523	0.137337	0.041265	0.183928
Age	0.544341	0.263514	0.239528	-0.113970
Outcome	0.221898	0.466581	0.065068	0.074752

	Insulin	BMI	DiabetesPedigreeFunction	\
Pregnancies	-0.073535	0.017683	-0.033523	
Glucose	0.331357	0.221071	0.137337	
BloodPressure	0.088933	0.281805	0.041265	
SkinThickness	0.436783	0.392573	0.183928	
Insulin	1.000000	0.197859	0.185071	
BMI	0.197859	1.000000	0.140647	
DiabetesPedigreeFunction	0.185071	0.140647	1.000000	
Age	-0.042163	0.036242	0.033561	
Outcome	0.130548	0.292695	0.173844	

	Age	Outcome
Pregnancies	0.544341	0.221898
Glucose	0.263514	0.466581
BloodPressure	0.239528	0.065068
SkinThickness	-0.113970	0.074752
Insulin	-0.042163	0.130548
BMI	0.036242	0.292695
DiabetesPedigreeFunction	0.033561	0.173844
Age	1.000000	0.238356
Outcome	0.238356	1.000000

Histograms of Iris Dataset





Conclusion

Thus, descriptive analysis was performed by reading data from different files and web sources using Python. Employee data was created and stored as a CSV file, and the Iris and Diabetes datasets were analyzed using correlation matrices. Histograms were also generated to visualize the distribution of the numerical features in both datasets.

Ex.No. 6	Working with Univariate and Bivariate Analysis
DATE: 08.10.2026	

Objective

To perform univariate and bivariate analysis on heart disease datasets obtained from two different sources and to:

Calculate frequency, mean, median, mode, and standard deviation.

Construct a linear regression model.

Perform multiple regression analysis.

Evaluate the performance of the regression models using Mean Squared Error (MSE).

Procedure

Download two heart disease datasets from different sources such as Kaggle and UCI.

Import the required Python libraries.

Load the two datasets into separate Pandas DataFrames.

Display the first few records of both datasets to verify the data.

Perform univariate analysis on the age attribute of each dataset.

Calculate:

Frequency

Mean

Median

Mode

Standard deviation

Select an independent variable and target variable for linear regression.

Split the dataset into training and testing sets.

Construct and train the linear regression model.

Predict the target values using the test data.

Calculate the Mean Squared Error (MSE) to evaluate the linear regression model.

Select multiple independent variables for multiple regression analysis.

Train the multiple regression model using the selected features.

Predict the target values using the test data.

Calculate the MSE for the multiple regression model.

1. Repeat the regression analysis for the second dataset.
2. Compare the performance of the regression models based on their MSE values.

Program

```
import pandas as pd import numpy as np
  from sklearn.model_selection import
  train_test_split from sklearn.linear_model
  import LinearRegression from sklearn.metrics
  import mean_squared_error from scipy import
  stats
  # Function for Univariate
  Analysis def
  univariate_analysis(df,
  column):
     freq =
     df[column].value_counts()
     mean = df[column].mean()
     median =
     df[column].median()
     mode = stats.mode(df[column],
     keepdims=True)[0][0] std = df[column].std()
     return freq, mean, median,
  mode, std # ----------------------------
  ----------
  # Load Heart Disease Dataset 1
```

```
# ----------------------------------
heart_df1 = pd.read_csv(
    "C:\\Users\\Renuka\\Desktop\\python
    programs\\heart.csv"
)
# Load Heart Disease Dataset 2
heart_df2 = pd.read_csv(
    "C:\\Users\\Renuka\\Desktop\\python  programs\\heart_failure_clinical_records_dataset.csv"
)
# Display dataset heads
print("Heart Disease
Dataset 1:")
print(heart_df1.head())
print("\nHeart Disease Dataset 2:")
print(heart_df2.head())
# ----------------------------------
# Univariate Analysis - Dataset 1
# ----------------------------------
column1 = 'age'
freq1, mean1, median1, mode1, std1
    = \ univariate_analysis(heart_df1,
    column1)
print("\nUnivariate Analysis for age in Dataset
1:") print("Frequency:")
print(freq1)
print("Mean:",
mean1)
print("Median:", median1)
print("Mode:", mode1)
```

```
print("Standard Deviation:",
std1)
# ---------------------------------
# Univariate Analysis - Dataset 2
# ---------------------------------
column2 = 'age'
freq2, mean2, median2, mode2, std2
   = \ univariate_analysis(heart_df2,
   column2)
print("\nUnivariate Analysis for age in Dataset
2:") print("Frequency:")
print(freq2)
print("Mean:",
mean2)
print("Median:", median2)
print("Mode:", mode2)
print("Standard Deviation:",
std2)
# ---------------------------------
# Bivariate Analysis - Simple Linear
Regression # Dataset 1
# ---------------------------------
X1 =
heart_df1[['age']]
y1 =
heart_df1['target']
X1_train, X1_test, y1_train, y1_test = \
   train_test_split(
```

```
        X1, y1,
        test_size=0.2,
        random_state=
        42
    )
lin_reg1 = LinearRegression()
lin_reg1.fit(X1_train, y1_train)
y1_pred =
lin_reg1.predict(X1_test)
mse1 = mean_squared_error(y1_test,
y1_pred) print("\nLinear Regression -
Dataset 1") print("Coefficient:",
lin_reg1.coef_[0]) print("Intercept:",
lin_reg1.intercept_) print("Mean Squared
Error:", mse1)
#  ---------------------------------
# Multiple Regression Analysis -
Dataset 1 # ---------------------------------
-----------------
features1
    = [ 'age',
    'trestbps
    ', 'chol',
    'thalach'
    ,
    'oldpeak
    '
]
X1_multi = heart_df1[features1]
```

```
y1_multi = heart_df1['target']

X1_train_multi, X1_test_multi, \
y1_train_multi, y1_test_multi =
train_test_split(
    X1_multi,
    y1_multi,
    test_size=0.
    2,
    random_stat
    e=42
)
multi_reg1 = LinearRegression()
multi_reg1.fit(
    X1_train_multi,
    y1_train_multi
)
y1_pred_multi =
    multi_reg1.predict(
    X1_test_multi
)
mse1_multi =
    mean_squared_error(
    y1_test_multi,
    y1_pred_multi
)
print("\nMultiple Regression -
Dataset 1") print("Coefficients:",
```

```
multi_reg1.coef_) print("Intercept:",
multi_reg1.intercept_) print("Mean
Squared Error:", mse1_multi) # ------
----------------------------------------
# Bivariate Analysis - Simple Linear
Regression # Dataset 2
# ----------------------------------
X2 = heart_df2[['age']]
y2 = heart_df2['DEATH_EVENT']
X2_train, X2_test, y2_train, y2_test = \
    train_test_split(
        X2, y2,
        test_size=0.2,
        random_state=
        42
    )
lin_reg2 = LinearRegression()
lin_reg2.fit(X2_train, y2_train)
y2_pred =
lin_reg2.predict(X2_test) mse2
= mean_squared_error(
    y2
    _t
    es
    t,
    y2
    _p
    re
    d
```

```
)
print("\nLinear Regression - Dataset
2") print("Coefficient:",



lin_reg2.coef_[0]) print("Intercept:",
lin_reg2.intercept_) print("Mean
Squared Error:", mse2)
#  ---------------------------------
# Multiple Regression Analysis -
Dataset 2 # ------------------------------
------------------
features2
    = [ 'age',
    'ejection_fraction',
    'serum_creatinine
    ', 'serum_sodium'
]
X2_multi = heart_df2[features2]
y2_multi =
heart_df2['DEATH_EVENT']
X2_train_multi, X2_test_multi, \
y2_train_multi, y2_test_multi =
train_test_split(
    X2_multi,
    y2_multi,
    test_size=0.
    2,
    random_stat
```

```
    e=42
)
multi_reg2 = LinearRegression()
multi_reg2.fit(
    X2_train_multi,
    y2_train_multi
)
y2_pred_multi =
    multi_reg2.predict(
    X2_test_multi
)
mse2_multi =
    mean_squared_error(
    y2_test_multi,
    y2_pred_multi
)
print("\nMultiple Regression -
Dataset 2") print("Coefficients:",
multi_reg2.coef_) print("Intercept:",
multi_reg2.intercept_) print("Mean
Squared Error:", mse2_multi)
```

Output

```
Heart Disease Dataset 1:
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  \
0   63    1   3       145   233    1        0      150      0      2.3      0
1   37    1   2       130   250    0        1      187      0      3.5      0
```

2	41	0	1	130	204	0	0	172	0	1.4	2
3	56	1	1	120	236	0	1	178	0	0.8	2
4	57	0	0	120	354	0	1	163	1	0.6	2

	ca	thal	target
0	0	1	1
1	0	2	1
2	0	2	1
3	0	2	1
4	0	2	1

Heart Disease Dataset 2:

	age	anaemia	creatinine_phosphokinase	diabetes	ejection_fraction	\
0	75.0	0	582	0	20	
1	55.0	0	7861	0	38	
2	65.0	0	146	0	20	
3	50.0	1	111	0	20	
4	65.0	1	160	1	20	

	high_blood_pressure	platelets	serum_creatinine	serum_sodium	sex	\
0	1	265000.00	1.9	130	1	
1	0	263358.03	1.1	136	1	
2	0	162000.00	1.3	129	1	
3	0	210000.00	1.9	137	1	
4	0	327000.00	2.7	116	0	

	smoking	time	DEATH_EVENT
0	0	4	1

1	0	6	1
2	1	7	1
3	0	7	1
4	0	8	1

Univariate Analysis for age in

Dataset 1: Frequency:

age

58	19
57	17
54	16
59	14
52	13
51	12
62	11
44	11
56	11
60	11
64	10
41	10
63	9
67	9
65	8
53	8
61	8
45	8
43	8
42	8
55	8
66	7

48 7

50 7

46 7

49 5

47 5

68 4

39 4

35 4

70 4

40 3

69 3

38 3

71 3

37 2

34 2

29 1

74 1

76 1

77 1

Name: count, dtype:

int64 Mean:

54.366336633663366

Median: 55.0

Mode: 58

Standard Deviation: 9.082100989837857

Univariate Analysis for age in

Dataset 2: Frequency:

age

60.000 33

50.000	27
65.000	26
55.000	17
75.000	11
53.000	10
58.000	10
63.000	8
80.000	7
42.000	7
72.000	7
40.000	7
85.000	6
62.000	5
52.000	5
68.000	5
61.000	4
59.000	4
49.000	4
73.000	4
51.000	4
90.000	3
64.000	3
69.000	3
46.000	3
48.000	2
78.000	2
54.000	2
44.000	2

82.000 2

57.000 2

67.000 2

66.000 2

60.667 2

83.000 1

87.000 1

79.000 1

41.000 1

94.000 1

86.000 1

81.000 1

43.000 1

47.000 1

56.000 1

Name: count, dtype:

int64 Mean:

60.837237458193975

Median: 60.0

Mode: 60.0

Standard Deviation: 11.900919367810586

Linear Regression - Dataset 1

Coefficient: -

0.012662926141768109

Intercept: 1.239245316316131

Mean Squared Error: 0.24118315432854578

Multiple Regression - Dataset 1

Coefficients: [-2.89305676e-03 -9.81088772e-04 -5.02214070e-05 5.62595498e-03
-1.55783175e-01]

Intercept: 0.16074075884463068

Mean Squared Error: 0.17927479585733166

Linear Regression - Dataset 2

Coefficient:

0.011049843534715878

Intercept: -0.37767827277566574

Mean Squared Error: 0.25852387400985954

Multiple Regression - Dataset 2

Coefficients: [0.00980361 -0.00995478 0.13019205 -0.00689752]

Intercept: 0.8396520809525791

Mean Squared Error: 0.22992407710085555

Conclusion

Thus, univariate and bivariate analysis was performed on two heart disease datasets. The frequency, mean, median, mode, and standard deviation were calculated for the selected attribute. Linear regression and multiple regression models were constructed, and their performance was evaluated using Mean Squared Error (MSE).